

Titalks

Project Proposal for UCS503P

Submitted to: Ms. Anushka

Thapar Institute of Engineering and Technology

Ananya, CSED¹

Hitesh, CSED²

Bhavya, CSED³

Pratham, CSED⁴

August 18, 2026

1 Higher-order goal

... secondary framing

This project aims to give verified TIET students a safer, more trustworthy digital campus by reducing reliance on scattered, unmoderated external apps for everyday campus life. While the topic is broader than any single feature, the immediate engineering objective is to translate *verified, moderated campus interaction* into a measurable, deployable software solution.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by shipping a synchronous core (sign-in, profiles, posts, feed) before layering in asynchronous, event-driven infrastructure.
- We will prioritize fast feedback over perfection by keeping unfinished features behind server-side flags and using CI-backed automation from the first iteration to reduce release friction.
- Success is defined by what can be delivered and verified in the MVP (Phases 0–2), then extended with messaging, Stories, and scale demonstrations based on what the pilot shows.

3 Problem Statement

Students at a large residential institute like TIET currently coordinate campus life across a patchwork of external, unverified channels (WhatsApp groups, Instagram pages, third-party apps). Common pain points include:

- **Fragmentation:** campus posts, meme pages, and club communities are spread across incompatible platforms, causing lost context and duplicated effort.
- **No verified membership:** open platforms cannot restrict participation to genuine, verified @thapar.edu accounts, which invites impersonation and outsider intrusion into campus discourse.
- **Unaccountable moderation:** reports made on external platforms have no institutional review path, no recorded decision, and no audit trail.
- **Poor discovery:** there is no single, campus-scoped place to find relevant content, communities, or ongoing conversations.

Why this matters now (contextual relevance): as student life increasingly happens online, an unverified and unmoderated sprawl across apps raises real safety and impersonation risks; even a lightweight, verified, moderated layer can noticeably improve trust and discoverability on campus.

¹Ananya, Roll No: 102403030299

²Hitesh, Roll No: 102403030298

³Bhavya, Roll No: 1024030304

⁴Pratham, Roll No: 1024030301

4 Proposed Solution

... *Software Proposition*

4.1 Overview

Build a mobile-first “**Titalks**” campus social platform restricted to verified TIET members, with the following components:

- **Verified onboarding:** Google sign-in restricted to accounts ending in `@thapar.edu`, with one profile per identity.
- **Social content:** posts and short vertical reels with likes, comments, saves, shares, and campus/followers/community visibility.
- **Feed and discovery:** ranked discovery, following, and community feeds with explainable, versioned ranking.
- **Communities:** student-requested, admin-approved communities with owners, moderators, and rules.
- **Messaging and Stories:** one-to-one and group messaging, and expiring Stories with highlights and polls.
- **Moderation and admin dashboard:** automated content screening followed by human review, with an auditable decision and appeals trail.

4.2 Core workflow

1. Member signs in with Google; the system verifies the `@thapar.edu` domain and creates a single profile for that identity.
2. Member publishes a post, reel, or Story, or joins a community and its feed.
3. New content passes automated moderation screening: clean content publishes immediately, flagged content stays private pending review.
4. An admin reviews the flagged queue and records a decision (approve, reject, warn, suspend); the member may appeal eligible decisions.
5. Member interacts with content and other members, and receives in-app notifications for follows, mentions, and decisions.

4.3 Operational constraints for an educational, zero-cost setting

- Run entirely on free/open-source software and free tiers; no paid subscription, licence, domain, or hosting plan is part of the mandatory baseline.
- Demonstrate the full distributed topology (Kafka, Valkey, replicas) locally rather than as always-on production infrastructure.
- Keep primary journeys usable on a phone-sized browser and installable as a Progressive Web App.

5 Solution Approach

... *Engineering focus*

This is an engineering course project for a four-person team in one semester, so we focus on a working synchronous core, contract-driven integration, and governance/moderation workflow rather than novel ranking or moderation-model research.

5.1 Moderation and quality assistance

... non-research

Moderation will use existing, free/open-source models behind one adapter rather than a custom-trained classifier:

- Apply an off-the-shelf visual NSFW classifier, a toxic-text classifier, OCR, and audio transcription to sampled content, without claiming state-of-the-art accuracy.
- Combine automated findings with a policy engine; only hold or flag content for human review, never fully auto-delete or auto-ban.
- Run the moderation logic in an isolated, non-internet-facing sidecar so it stays swappable and testable behind a deterministic mock.

5.2 Web architecture

... web-based solution

A feature-based modular monolith, not premature microservices:

- **Frontend:** a mobile-first, installable web app for members and admins (feed, submission, dashboard, status screens).
- **Backend API:** one controlled application API handling authentication, content, feed, and moderation logic.
- **Data, identity, and storage:** a self-hosted managed-database platform providing authentication, relational storage, object storage, and realtime transport.
- **Async layer:** an event stream and an in-memory cache/rate-limit layer, introduced once the synchronous core is stable.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically lint, type-check, and run unit and integration tests (against disposable database, event, and cache services) on every change.
- Produce repeatable, versioned deployment artifacts to staging, including dependency, secret, and container security scans.
- Enable safe and frequent releases of incremental features, with unfinished work kept behind server-side flags.

6 Evaluation Criterion

... measurable and attributable

We define success using measurable metrics tied to the core moderation and social workflow.

6.1 Primary evaluation metric

Time-to-moderation-decision (TTMD): time between a piece of content or a report entering the moderation queue and an admin recording a decision on it.

- **Target:** median TTMD within the pilot window at a level the team defines and instruments before development begins.
- **Attribution:** measured from database timestamps (flag/report creation versus recorded decision by an admin).

6.2 Secondary evaluation metrics

- **Onboarding integrity:** percentage of sign-in attempts from non-@thapar.edu accounts correctly rejected.
- **Feed responsiveness:** API latency for cached and non-cached feed reads, measured against the project’s own performance targets.
- **Moderation precision:** proportion of automatically flagged, otherwise-clean content overturned on human review.
- **Privacy isolation:** confirmation that blocked or unrelated members cannot retrieve private content or messages, verified through authorization tests.
- **Reliability:** loss of the event stream, cache, or a worker degrades optional features without corrupting authoritative records.

6.3 Pilot validation plan ... high-level

- Recruit a small cohort of verified pilot student members and a handful of student admins acting as moderators.
- Seed campus-relevant posts and communities; compare moderation turnaround and feed latency against a logged baseline within the iteration window.
- Run scripted resilience drills (deliberately stopping the event stream, cache, or a worker) during the pilot to confirm data integrity is preserved.

7 Scalability

... with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** the system should support growth in registered members and concurrent realtime connections by:
 - decoupling the frontend and backend behind versioned contracts,
 - using a stateless API design, and
 - introducing a database read replica once the synchronous core is proven.
2. **Operationally deployable (practically):** the system should run on common, largely free infrastructure:
 - a managed or self-hosted database/identity/storage platform,
 - containerized deployment, and
 - a CI/CD pipeline for staging and production.

8 Engine availability heuristic

A mature set of free/open-source “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for a mobile-first frontend and a REST application API.
- Self-hosted managed platform for authentication, relational database, object storage, and realtime transport.

- Event stream for asynchronous processing once the synchronous core is stable.
- In-memory store for caching, rate limits, and locks.
- Open-source malware scanning and media-processing libraries.
- Off-the-shelf, locally runnable models for visual, text, and audio content screening.
- Metrics, dashboards, and tracing tooling for observability.
- A CI runner with an equivalent local command for every check.

We will use these mature components to focus on workflow, correctness, safety, and measurement rather than inventing new infrastructure or training new models.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Verified Google sign-in restricted to @thapar.edu, with one profile per identity
- Posts and reels with campus/followers/community visibility
- Discovery and following feed with an initial deterministic ranking
- Communities: student request and admin approval workflow
- Basic reporting and a moderation-status pipeline with an admin review screen
- CI: build, lint, and backend unit/integration tests against disposable services

9.2 Subsequent deliverables

- Event-driven architecture (event stream, cache, workers) and the full moderation adapter (visual, text, OCR, transcription)
- Messaging (one-to-one and group) with realtime delivery, receipts, and moderation
- Stories with expiry, highlights, replies, and polls
- Read replica, full local distributed topology, dashboards, and resilience drills

10 Risks and mitigations

- **Scope risk:** four students, one semester — ship the core social and moderation MVP first; stage messaging, Stories, and advanced ranking for later iterations.
- **Integration risk:** parallel work across four owners — agree on API/event contracts and database migrations before implementation branches diverge.
- **Privacy and safety risk:** verified-only membership and private content — enforce authentication, database-level row security, private storage, and authorization tests.
- **Moderation false positives:** free, local models are not commercial-grade — keep human review, recorded reasons, and an appeals path in every decision.
- **Free-tier limits:** quotas may run out mid-semester — run the full stack in local containers with deterministic mocks instead of purchasing capacity.
- **Knowledge concentration:** one member may carry most integration risk — assign a secondary reviewer and keep documentation current for every subsystem.

11 Summary

This project proposes a verified, zero-cost campus social platform for TIET that gives students one trustworthy, moderated space for campus life in place of an unmanaged sprawl across external apps. It meets the course criteria by sequencing delivery around a synchronous MVP core before asynchronous, event-driven scale work, using CI/CD to support frequent, safe releases, and defining measurable evaluation criteria (especially moderation turnaround and feed responsiveness). It remains focused on engineering workflow, safety, and scalability readiness rather than research-driven novelty in ranking or moderation.